AI-BASED STUDENT PERFORMANCE PREDICTION SYSTEM FOR EARLY IDENTIFICATION OF AT-RISK STUDENTS USING MACHINE LEARNING

A Capstone / Course Project Report submitted in partial fulfilment of the requirements for the course

CSA1711 – ARTIFICIAL INTELLIGENCE

towards the award of the degree of

BACHELOR OF TECHNOLOGY

IN

CSE(DS)

Submitted by

T. Niranjan Kumar (192473007)

Under the Supervision of

Dr. Krishnakumar Kathiresan

SAVEETHA ENGINEERING COLLEGE

SEPTEMBER 2026

SAVEETHA ENGINEERING COLLEGE

[Insert City / Address]

BONAFIDE CERTIFICATE

This is to certify that the project entitled "AI-BASED STUDENT PERFORMANCE PREDICTION SYSTEM FOR EARLY IDENTIFICATION OF AT-RISK STUDENTS USING MACHINE LEARNING" has been carried out by T. Niranjan Kumar ([Register No.]) under the supervision of Dr. Krishnakumar Kathiresan, and is submitted in partial fulfilment of the requirements for CSA1711 – ARTIFICIAL INTELLIGENCE at SAVEETHA ENGINEERING COLLEGE.

COURSE FACULTY

Dr. Krishnakumar Kathiresan

SAVEETHA ENGINEERING COLLEGE,

DECLARATION

I, T. Niranjan Kumar ([Register No.]), a student of BACHELOR OF TECHNOLOGY in [Insert Department / Branch Name], SAVEETHA ENGINEERING COLLEGE, hereby declare that the project work entitled "AI-BASED STUDENT PERFORMANCE PREDICTION SYSTEM FOR EARLY IDENTIFICATION OF AT-RISK STUDENTS USING MACHINE LEARNING" is the result of my own bonafide effort. To the best of my knowledge, the work presented here is original and has been prepared in accordance with academic integrity guidelines. All datasets, libraries, tools and reference material used in the project have been appropriately acknowledged. Any AI-assisted tools used during development have been used as supporting resources only and have been acknowledged accordingly.

Place: [City]

Date: [Date]

Signature of the Student

T. Niranjan Kumar (192473007)

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Responsibilities	Report Contribution	Contribution (%)
P. Shashank ([Register No.])	Requirement analysis, data collection and preprocessing pipeline (Module 1), dataset validation and cleaning	Introduction, Literature Review, Module 1 documentation	34%
T. Niranjan Kumar ([Register No.])	Model development – Random Forest and XGBoost training, evaluation and comparison (Module 2)	Engineering Design, Module 2 documentation, Results	33%
P. Abhinay ([Register No.])	Web dashboard design and deployment, individual and batch prediction workflow (Module 3)	Module 3 documentation, Testing, Conclusion	33%
Total			100%

ABSTRACT

Academic performance is shaped by a combination of academic and behavioural factors such as attendance, marks, assignment completion and study hours, which makes it difficult and time-consuming for teaching staff to monitor every student manually and identify weak or at-risk learners early. This project addresses the engineering problem of predicting a student's academic performance in advance and flagging at-risk students using historical academic and behavioural data. The proposed system applies supervised machine learning – specifically Random Forest and XGBoost classifiers – to model the relationship between recorded student features and performance outcomes, and compares the two algorithms using standard, measurable evaluation criteria: accuracy, precision, recall and F1-score. The better-performing model is selected, persisted, and served through a web-based dashboard that supports dataset exploration, model comparison, an analytics overview, individual student prediction and batch prediction with a downloadable results file.

The system is designed as a decision-support tool for teachers and academic coordinators rather than an autonomous decision-maker: it classifies students into risk categories and reports the contributing factors behind each prediction so that staff retain judgement over any intervention. Functional verification in this report covers data preprocessing correctness, model training and evaluation behaviour, and end-to-end dashboard functionality for both individual and batch prediction. Where large-scale longitudinal outcome data was not available at the time of writing, the report avoids unsupported generalization claims and treats extended real-world validation as future work. The outcome is a modular, explainable and extensible prototype that connects academic and behavioural evidence to timely, actionable support for at-risk students.

Keywords: Machine Learning, Random Forest, XGBoost, Student Performance Prediction, Educational Data Mining, Risk Classification, Decision Support, Web Dashboard

TABLE OF CONTENTS

Section	Page
Bonafide Certificate	2
Declaration	3
Individual Contribution Statement	4
Abstract	5
List of Figures	8
List of Tables	9
List of Abbreviations	10
CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	11
1.1 Background	11
1.2 Problem Statement	11
1.3 Engineering Problem Definition	11
1.4 Project Aim	12
1.5 Project Objectives	12
1.6 Scope	12
1.7 Expected Engineering Outcome	13
CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	14
2.1 Review Method	14
2.2 Review of Existing Work	14
2.3 Comparative Analysis	14
2.4 Research / Engineering Gap	15
2.5 Proposed Contribution	15
CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	16
3.1 Requirements	16

Section	Page
3.2 Applicable Standards and References	17
3.3 Design Specifications	17
3.4 Alternative Solution Evaluation	17
3.5 System Architecture	18
3.6 Trade-off Analysis	18
3.7 Sustainability, Ethics, Safety, Risk & Security	19
CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	21
4.1 Development Tools and Resources	21
4.2 Module 1: Data Collection & Preprocessing (Summary)	21
4.3 Module 2: Model Development — Detailed Implementation	21
4.4 Module 3: Dashboard & Deployment — Detailed Implementation	24
4.5 Testing	25
4.6 Results and Discussion	26
4.7 Achievement of Objectives	26
4.8 Project Management	26
CHAPTER 5: CONCLUSION & REFLECTION	28
5.1 Conclusion	28
5.2 Limitations	28
5.3 Future Scope	28
5.4 Reflection	29
REFERENCES	30
APPENDICES	31
Appendix A – Glossary of Terms	31
Appendix B – Additional Functional Requirements / User Stories	31

Section	Page
Appendix C – Detailed Test Cases	31
Appendix D – Data Dictionary	32
Appendix E – Ethics and Data-Handling Checklist	32
Appendix F – Individual Contribution Evidence	32
Appendix G – Outcome Mapping Sheet	33

LIST OF FIGURES

Figure No.	Figure Name
Figure 1	Overall system architecture (Module 1 → Module 2 → Module 3)
Figure 2	Data preprocessing and feature preparation workflow
Figure 3	Model training and comparison workflow (Random Forest vs XGBoost)
Figure 4	Web dashboard – analytics and model comparison view
Figure 5	Individual and batch student prediction workflow

LIST OF TABLES

Table No.	Table Name
Table 1	Comparative analysis of existing approaches
Table 2	Stakeholder and user requirements
Table 3	Functional and non-functional requirements
Table 4	Applicable standards and references
Table 5	Design specifications
Table 6	Alternative solution evaluation
Table 7	Sustainability, ethics, safety and security analysis
Table 8	Risk register
Table 9	Development tools and resources
Table 10	Model comparison – Random Forest vs XGBoost
Table 11	Test plan and verification
Table 12	Achievement of objectives
Table 13	Project timeline

LIST OF ABBREVIATIONS

Abbreviation	Description
ML	Machine Learning
RF	Random Forest
XGBoost	Extreme Gradient Boosting
EDA	Exploratory Data Analysis
EDM	Educational Data Mining
F1	F1-Score (harmonic mean of precision and recall)
API	Application Programming Interface
UI/UX	User Interface / User Experience

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Student academic performance is influenced by a wide combination of academic and behavioural factors, including attendance, internal and external marks, assignment completion and study hours. Teaching staff traditionally track these factors manually across spreadsheets, mark registers and attendance sheets, which becomes difficult and time-consuming as class sizes and the number of tracked parameters grow. As a result, students who are gradually falling behind are often identified late, after several assessment cycles have already been missed, by which point corrective action is less effective.

Machine learning offers a practical way to address this gap: supervised learning algorithms can identify patterns in historical student records and generalize those patterns to predict outcomes for new or current students. Ensemble tree-based methods, in particular Random Forest and gradient-boosted trees such as XGBoost, are well suited to structured, tabular educational data because they handle non-linear relationships between features, are relatively robust to noisy or missing values, and can report feature importance, which supports explainability – an important requirement when predictions are used to guide decisions about real students.

1.2 Problem Statement

Student performance is difficult to monitor manually, and weak or at-risk students are frequently not identified early enough for timely academic support. Existing academic-record systems store the raw data – attendance, marks, assignment and study-hour records – but do not automatically convert this data into an accurate, forward-looking prediction of a student's performance or risk level. There is a need for an AI-based system that can learn from historical academic and behavioural data, predict performance for new students, classify them by risk level, and present the contributing factors and results through an accessible web interface for teachers and academic coordinators.

1.3 Engineering Problem Definition

The engineering problem is to design and implement a web-based machine-learning system that ingests raw academic and behavioural student data, transforms it into a clean and consistent

feature set, trains and compares multiple classification models, and serves the best-performing model through a dashboard capable of individual and batch prediction – while remaining interpretable, testable and usable by non-technical staff.

The problem involves the following considerations, wherever applicable:

- The system must handle raw, potentially inconsistent academic and behavioural data (missing values, categorical fields, mismatched column formats) before it can be used for model training.
- Multiple candidate algorithms (Random Forest and XGBoost) introduce a trade-off between predictive accuracy, training/inference cost and interpretability that must be evaluated using objective metrics rather than assumption.
- Higher predictive accuracy must be balanced against explainability, fairness and the risk of a prediction being misread as a certain outcome for a real student.
- The prototype is constrained by the size, quality and representativeness of the available academic dataset, and by the fact that longitudinal, real-world outcome data may not yet exist to validate long-term predictive accuracy.

1.4 Project Aim

To design and implement a web-based AI system that predicts student academic performance from historical academic and behavioural data using Random Forest and XGBoost, compares the models using standard evaluation metrics, and presents predictions, risk levels and contributing factors through an interactive dashboard to support early identification of at-risk students.

1.5 Project Objectives

1. Design a modular pipeline covering data collection, preprocessing, model development and web-based deployment.
2. Build a preprocessing workflow that validates dataset columns, handles missing values, encodes categorical fields and separates identifying fields (Student ID, Name) from machine-learning features.
3. Train and compare a Random Forest classifier and an XGBoost classifier on the prepared dataset using a proper train/test split.

4. Evaluate both models using accuracy, precision, recall and F1-score, and select and persist the best-performing model.
5. Implement a web-based dashboard for dataset exploration, model comparison, an analytics overview, individual student prediction and batch prediction with downloadable results.
6. Test the system across preprocessing, model training/evaluation and dashboard workflows, and document limitations and future validation needs.

1.6 Scope

Included

- Collection and preprocessing of academic and behavioural student data (attendance, marks, assignments, study hours and related fields).
- Training, evaluation and comparison of Random Forest and XGBoost classification models.
- Selection and persistence (saving) of the best-performing trained model.
- A web dashboard for dataset exploration, model comparison, analytics, individual prediction and batch prediction with downloadable results.
- Functional and integration testing of the preprocessing, modelling and dashboard components.

Excluded

- Automated, unsupervised academic decisions (e.g., grade changes, admission or scholarship decisions) made without human review.
- Clinical, psychological or disciplinary assessment of any student.
- Claims that model predictions are equivalent to a qualified teacher's professional judgement.
- Continuous, real-time data ingestion from live institutional systems (treated as future scope).

Assumptions

- A representative academic and behavioural dataset (attendance, marks, assignments, study hours and a performance label) is available for training and testing.
- Column names and data formats used by the institution can be validated and standardized during preprocessing.
- Teachers and academic coordinators are the intended users of the dashboard and retain final decision-making authority.

Boundary Conditions

- The platform is a prototype decision-support tool, not an autonomous evaluator of students.
- Model accuracy depends on dataset size, quality and representativeness, and may vary across cohorts.
- Student data must be handled with appropriate privacy and access-control safeguards.

1.7 Expected Engineering Outcome

The intended engineering outcome is a working software prototype consisting of:

- A reusable data preprocessing pipeline that cleans and encodes raw academic and behavioural data.
- Two trained and compared classification models (Random Forest and XGBoost) with a documented, metric-based selection process.
- A persisted best-performing model ready for inference.
- A web-based dashboard offering dataset exploration, model comparison, analytics, individual prediction and batch prediction with a downloadable results file.
- A modular architecture in which the data, modelling and presentation components can each be tested and evaluated independently.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review was organized around established research in educational data mining, ensemble machine learning and applied data-science tooling. Foundational work on ensemble decision trees, gradient boosting, general-purpose machine-learning libraries and data-analysis tooling was used as a reference point to justify the algorithm and tooling choices made in this project, rather than to claim reproduction of any specific published system.

2.2 Review of Existing Work

The review considered the following technical areas:

- Educational data mining and learning-analytics research that applies statistical and machine-learning techniques to academic records to identify at-risk students.
- Ensemble decision-tree methods, particularly Random Forest, which combine multiple decision trees to improve accuracy and reduce overfitting compared with a single tree (Breiman, 2001).
- Gradient-boosted tree methods, particularly XGBoost, which build trees sequentially to correct prior errors and are widely used for structured/tabular prediction tasks (Chen & Guestrin, 2016).
- General-purpose machine-learning libraries that provide reusable implementations of classification algorithms and evaluation metrics (Pedregosa et al., 2011).
- Data-handling and analysis tooling used to prepare structured datasets for modelling (McKinney, 2010).
- Deployment considerations for serving trained ML models through accessible dashboards for non-technical end users.

2.3 Comparative Analysis

Table 1. Comparative analysis of existing approaches

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual teacher monitoring	Uses direct classroom knowledge of the student	Time-consuming, inconsistent across teachers, and detects issues late	Motivates an automated, earlier-warning system
Rule-based / threshold scoring (e.g., attendance below X%)	Simple, transparent and easy to explain	Cannot capture interactions between multiple factors; brittle across cohorts	Useful as an interpretable baseline for comparison
Single-algorithm ML models (e.g., logistic regression or a single decision tree)	Lightweight, fast to train and easy to interpret	Often lower accuracy on non-linear, multi-factor academic data	Justifies comparing against a stronger ensemble model
Ensemble tree-based ML (Random Forest, XGBoost) – proposed approach	Higher predictive accuracy on structured/tabular data; supports feature-importance explainability	Requires labelled historical data and careful evaluation to avoid overfitting	Selected as the core modelling approach for this project

2.4 Research / Engineering Gap

The main engineering gap is the absence of a practical, end-to-end workflow that connects raw academic and behavioural data capture, preprocessing, model comparison and an accessible prediction interface in a single institution-usable application. Individual techniques – data cleaning, Random Forest classification, XGBoost classification – are each well studied, but they do not automatically provide a ready-to-use pipeline that a teacher or academic coordinator can operate without programming knowledge.

Open issues that remain include limited availability of large, labelled, representative institutional datasets; the risk of a prediction being over-trusted as a certain outcome rather than a probabilistic estimate; the need to keep the system explainable so that predictions can be questioned and reviewed; and the absence, at this stage, of longitudinal outcome data to

validate whether early interventions triggered by the system actually improve student outcomes.

2.5 Proposed Contribution

The proposed contribution is an integrated student-performance prediction prototype that connects data preprocessing, comparative model training (Random Forest and XGBoost) and a web-based dashboard for individual and batch prediction. The project treats usability, explainability, testing and responsible use of predictions as engineering requirements rather than afterthoughts, and positions the system explicitly as a decision-support tool that assists – rather than replaces – the professional judgement of teachers and academic coordinators.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Table 2. Stakeholder and user requirements

Stakeholder	Requirement
Teacher / Academic coordinator	Early, interpretable warning of at-risk students with the contributing factors behind each prediction
Student	Fair and consistent treatment; predictions used to support, not label, their progress
System administrator	A modular, testable and maintainable web application that is straightforward to deploy and update
Institution / Assessor	Evidence that predictions are validated using proper train/test evaluation and standard metrics

Functional & Non-Functional Requirements

Table 3. Functional and non-functional requirements

Requirement	Type	Measurable Target
Dataset validation & preprocessing	Functional	Reject or flag datasets with missing required columns; handle missing values without crashing
Model training (Random Forest & XGBoost)	Functional	Both models train successfully on the prepared dataset and produce predictions on held-out test data
Model evaluation & comparison	Functional	Report Accuracy, Precision, Recall and F1-score for each model and identify the better-performing one

Requirement	Type	Measurable Target
Individual student prediction	Functional	Return a performance prediction, risk level and contributing factors for a single student record
Batch prediction	Functional	Process a batch file of multiple students and return downloadable results
Usability	Non-Functional	Dashboard usable by non-technical staff without additional training beyond a short walkthrough
Performance	Non-Functional	Individual prediction returned within a few seconds on a typical dataset size
Security & Privacy	Non-Functional	Student data access restricted to authorized dashboard users; no data exposed via public endpoints
Maintainability	Non-Functional	Modular separation between preprocessing, modelling and dashboard code

3.2 Applicable Standards and References

Table 4. Applicable standards and references

Area	Standard / Practice Followed
Machine learning implementation	scikit-learn API conventions for model training and evaluation (Pedregosa et al., 2011)
Ensemble modelling	Random Forest methodology (Breiman, 2001)
Gradient boosting	XGBoost scalable tree-boosting methodology (Chen & Guestrin, 2016)
Data handling	Tabular data-processing conventions (McKinney, 2010)
Software engineering practice	Modular design, version control and documented testing for the codebase
Data privacy	Institutional data-handling policy for storing and processing student records (institution-specific; to be confirmed with the institution)

3.3 Design Specifications

Table 5. Design specifications

Component	Specification
Input data	Structured dataset (CSV) containing Student ID, Name, attendance, marks, assignment and study-hour fields, plus a performance/label column for training
Preprocessing	Column validation, missing-value handling, categorical encoding, separation of identifier fields from ML features
Models	Random Forest Classifier and XGBoost Classifier (scikit-learn / xgboost implementations)
Train/test split	Stratified split (e.g., 80% training / 20% testing) to preserve class balance
Evaluation metrics	Accuracy, Precision, Recall, F1-score
Model persistence	Best-performing model saved (e.g., via joblib/pickle) for reuse by the dashboard
Dashboard	Web interface for dataset exploration, model comparison, analytics, individual prediction and batch prediction
Output	Predicted performance category, risk level, contributing factors, and a downloadable batch-results file

3.4 Alternative Solution Evaluation

Table 6. Alternative solution evaluation

Alternative	Why Considered	Why Not Selected (or Selected)
Manual / rule-based scoring only	Simple and fully transparent	Cannot model interactions between multiple factors; not selected as the sole method, but retained conceptually as an interpretable comparison point
Single decision tree	Very easy to visualize and explain	Prone to overfitting and lower accuracy than ensemble methods; not selected

Alternative	Why Considered	Why Not Selected (or Selected)
Logistic regression	Simple, fast, well-understood baseline	Limited ability to capture non-linear relationships common in behavioural data; considered but not the primary model
Random Forest	Robust ensemble method, resistant to overfitting, provides feature importance	Selected as one of the two compared models
XGBoost	State-of-the-art gradient boosting, typically strong accuracy on tabular data	Selected as the second compared model; best of the two is chosen after evaluation
Deep neural network	Can model complex patterns given enough data	Requires larger datasets and reduces interpretability; not selected for this prototype

3.5 System Architecture

The system is organized into three cooperating modules that mirror the project's engineering workflow: Module 1 (Data Collection & Preprocessing), Module 2 (Model Development) and Module 3 (Dashboard & Deployment). Raw academic and behavioural records enter through Module 1, where columns are validated, missing values handled, categorical fields encoded and identifier fields separated from the machine-learning feature set. The cleaned, labelled dataset is passed to Module 2, where it is split into training and testing partitions, used to train Random Forest and XGBoost classifiers, evaluated with accuracy, precision, recall and F1-score, and the best-performing model is selected and saved. Module 3 loads the saved model and exposes it through a web-based dashboard that supports dataset exploration, model comparison, an analytics view, and both individual and batch student prediction with downloadable results. Each module can be tested and evaluated independently, which keeps the pipeline modular and maintainable.

[Insert Figure / Screenshot Here]

Figure 1. Overall system architecture (Module 1 → Module 2 → Module 3)

3.6 Trade-off Analysis

Design Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Model choice	Single model (RF only or XGBoost only)	Simpler pipeline, fewer components to maintain	Risk of relying on a model that happens to underperform on this dataset	Train and compare both models; select the empirically better one using standard metrics
Interpretability vs. accuracy	Deep learning model	Potentially higher accuracy with enough data	Harder to explain individual predictions to teachers; needs more data than currently available	Use tree-based ensembles (RF, XGBoost) which support feature-importance explanations
Deployment style	Batch-only offline reports	Simple to build	Less convenient for day-to-day use by staff	Provide an interactive web dashboard with both individual and batch prediction

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 7. Sustainability, ethics, safety and security analysis

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Prefer lightweight, reusable software components over hardware-intensive infrastructure	No dedicated hardware required; the models can be trained and served on standard institutional computing resources	Reduces resource dependence and supports easy reuse across cohorts and departments
Ethics	Fairness, transparency and human oversight in how predictions are used	Risk that a prediction could be treated as a label rather than a probabilistic estimate; risk of bias if the training data under-represents certain student groups	Predictions are presented as decision-support evidence with contributing factors, not as a final judgement; teacher review is required before any intervention

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Safety	Avoid discouraging or stigmatizing students through automated labels	Risk levels (e.g., low/medium/high) could be misread as fixed traits	Risk categories are framed as indicators for support, with wording reviewed to remain constructive
Security	Protect student academic and behavioural data	Datasets and predictions may contain personally identifiable and sensitive academic information	Access to the dashboard and underlying data is restricted to authorized staff; security treated as a core non-functional requirement

Risk Register

Table 8. Risk register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Missing or inconsistent data fields	Medium	Medium	Medium	Column validation and missing-value handling in Module 1; reject clearly invalid datasets	Low-Medium
Model overfitting to training data	Medium	Medium	Medium	Proper train/test split, cross-validation where feasible, comparison across two algorithms	Low
Prediction bias against under-represented student groups	Medium	High	High	Review dataset balance; monitor performance across subgroups where possible; keep predictions advisory	Medium
Sensitive student data exposure	Low-Medium	High	High	Access control on the dashboard, minimization of displayed personal data, secure storage	Low-Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Over-reliance on automated predictions	Medium	High	High	Present predictions with contributing factors and confidence context; require human review before intervention	Medium
Integration failure between modules	Low-Medium	Medium	Medium	Module-level testing and a defined interface (saved model file + prediction API) between Module 2 and Module 3	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Tools and Resources

Table 9. Development tools and resources

Category	Tool / Library	Purpose
Programming language	Python 3.x	Core implementation language for preprocessing, modelling and backend logic
Data handling	pandas, NumPy	Loading, cleaning and transforming the academic/behavioural dataset
Machine learning	scikit-learn (RandomForestClassifier), xgboost (XGBClassifier)	Model training, evaluation and metric computation
Model persistence	joblib / pickle	Saving and loading the trained, best-performing model
Web dashboard	Flask / Streamlit (select one for implementation), HTML, CSS, JavaScript	Serving the interactive dashboard and prediction endpoints
Visualization	Matplotlib / Plotly	Analytics charts and model-comparison visuals in the dashboard
Development environment	Jupyter Notebook, VS Code	Iterative development, experimentation and debugging
Version control	Git	Source-code tracking and revision history for the project

4.2 Module 1: Data Collection & Preprocessing (Summary)

Module 1 collects student academic and behavioural data (attendance, marks, assignment scores, study hours and related fields) and prepares it for modelling. The module validates that required dataset columns are present, handles missing values (e.g., imputation or row exclusion depending on the field), encodes categorical variables into numeric form, and separates non-

predictive identifier fields (Student ID, Name) from the feature set used for training. The output of Module 1 is a clean, consistently formatted feature table with an associated performance label, ready for Module 2. Module 1 is treated as a prerequisite pipeline in this report; the remainder of this chapter focuses in depth on Module 2 and Module 3, which form the core machine-learning and delivery components of the system.

4.3 Module 2: Model Development — Detailed Implementation

Module 2 is responsible for turning the preprocessed dataset into a validated, deployable prediction model. It covers dataset splitting, training two candidate classifiers, generating predictions on unseen data, computing evaluation metrics, comparing the models, and selecting and saving the best-performing one.

4.3.1 Train/Test Split

The labelled dataset produced by Module 1 is divided into a training partition and a testing partition, with the testing partition held out and never used during training. A stratified split (for example, 80% training / 20% testing) is used so that the proportion of each performance/risk category is preserved in both partitions, which is important because student performance categories are often imbalanced (fewer students in the “high-risk” category than in “low-risk”).

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

4.3.2 Random Forest Model Training

A Random Forest classifier is trained on the training partition. Random Forest builds a large collection (“forest”) of decision trees on bootstrapped subsets of the training data and random subsets of features, then aggregates their individual predictions by majority vote. This reduces the overfitting risk associated with a single decision tree and typically improves generalization on structured, tabular data such as academic records.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=200, max_depth=None,
```

```
    criterion='gini', random_state=42
)
rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_test)
```

The trained Random Forest also exposes feature_importances_, which is used to report the contributing academic and behavioural factors behind each prediction – a key explainability requirement identified in Chapter 3.

4.3.3 XGBoost Model Training

An XGBoost classifier is trained on the same training partition as a second candidate model. XGBoost builds trees sequentially, with each new tree focused on correcting the errors made by the previous ones (gradient boosting), which often yields strong predictive performance on tabular datasets similar in structure to the student dataset used here.

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=200, learning_rate=0.1, max_depth=6,
    objective='multi:softmax', eval_metric='mlogloss',
    random_state=42
)
xgb_model.fit(X_train, y_train)
xgb_predictions = xgb_model.predict(X_test)
```

4.3.4 Evaluation Metrics

Both models are evaluated on the held-out test set using four standard classification metrics:

- Accuracy – the proportion of test samples correctly classified overall: Accuracy = (TP + TN) / (TP + TN + FP + FN).
- Precision – of the students predicted to be in a given risk category, the proportion that actually belong to it: Precision = TP / (TP + FP).
- Recall – of the students who actually belong to a given risk category, the proportion correctly identified: Recall = TP / (TP + FN).
- F1-score – the harmonic mean of precision and recall, useful when class distribution is imbalanced: F1 = 2 × (Precision × Recall) / (Precision + Recall).

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

def evaluate(y_test, y_pred):
    return {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, average='weighted'),
        'Recall': recall_score(y_test, y_pred, average='weighted'),
        'F1 Score': f1_score(y_test, y_pred, average='weighted'),
    }
```

4.3.5 Model Comparison and Selection

The two trained models are compared side-by-side on the same test set using the metrics above, and the model with the stronger overall balance of accuracy, precision, recall and F1-score is selected for deployment. The table below is provided as the reporting template used to record this comparison; the placeholder cells should be completed with the actual metric values obtained from running the notebook / script on the project dataset.

Table 10. Model comparison – Random Forest vs XGBoost

Metric	Random Forest	XGBoost
Accuracy	[Insert value]	[Insert value]
Precision	[Insert value]	[Insert value]
Recall	[Insert value]	[Insert value]
F1-Score	[Insert value]	[Insert value]
Training time	[Insert value]	[Insert value]
Selected model	[Mark ✓ if selected]	[Mark ✓ if selected]

(Replace the bracketed placeholders with your actual experimental results before submission — this report does not fabricate specific numeric outcomes that were not provided.)

4.3.6 Saving the Trained Model

Once the better-performing model is identified, it is serialized to disk so that Module 3 can load it without retraining. This decouples model development from deployment and allows the dashboard to serve predictions quickly.

```
import joblib
```

```
best_model = xgb_model  # or rf_model, based on evaluation
joblib.dump(best_model, 'best_student_performance_model.pkl')
```

[Insert Figure / Screenshot Here]

Figure 3. Model training and comparison workflow (Random Forest vs XGBoost)

4.4 Module 3: Dashboard & Deployment — Detailed Implementation

Module 3 delivers the trained model to end users through a web-based dashboard. It is designed for teachers and academic coordinators who are not expected to write code, so the interface emphasizes clear visual summaries, straightforward forms for individual prediction, and a simple upload/download flow for batch prediction.

4.4.1 Dashboard Features

- Dataset exploration – lets staff browse and filter the underlying academic/behavioural dataset used for training, to build trust in what the model has learned from.
- Model comparison – displays the Random Forest vs XGBoost metrics from Module 2 in a readable chart/table so users understand why a given model was chosen.
- Analytics dashboard – summary visuals (e.g., distribution of risk categories, most influential contributing factors) drawn from recent predictions.
- Individual student prediction – a form where a staff member enters or selects one student's academic/behavioural data and receives a predicted performance category, risk level and contributing factors.
- Batch prediction – allows uploading a CSV of multiple students; the system runs the saved model over the whole file and returns predictions for every row.
- Results and downloadable prediction – batch results (and individual results, where relevant) can be exported/downloaded as a CSV for record-keeping or further review.

4.4.2 Technology Stack and Architecture

The dashboard is implemented as a lightweight Python web application (Flask or Streamlit) that loads the model saved in Module 2 at startup and exposes it through simple routes/pages. A typical request flow for individual prediction is: the user submits a form → the backend applies the same preprocessing/encoding used in Module 1 to the submitted values → the saved

model produces a prediction → the result (performance category, risk level, contributing factors) is rendered back to the user.

```
# Simplified Flask-style prediction route
import joblib, pandas as pd

model = joblib.load('best_student_performance_model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    student_data = request.form.to_dict()
    features_df = preprocess_single_record(student_data)
    prediction = model.predict(features_df)[0]
    risk_level = map_to_risk_level(prediction)
    factors = get_top_contributing_factors(model, features_df)
    return render_template('result.html', prediction=prediction,
                           risk_level=risk_level, factors=factors)
```

4.4.3 Batch Prediction Workflow

For batch prediction, the user uploads a CSV file containing multiple student records in the same column format validated by Module 1. The backend applies the same preprocessing pipeline to the entire file, runs the saved model across all rows, appends a predicted performance category and risk level to each row, and offers the enriched file back to the user as a downloadable CSV.

```
@app.route('/batch_predict', methods=['POST'])
def batch_predict():
    uploaded_file = request.files['dataset']
    df = pd.read_csv(uploaded_file)
    features_df = preprocess_batch(df)
    df['Predicted_Performance'] = model.predict(features_df)
    df['Risk_Level'] = df['Predicted_Performance'].apply(map_to_risk_level)
    return send_file(to_downloadable_csv(df), as_attachment=True)
```

[Insert Figure / Screenshot Here]

Figure 4. Web dashboard – analytics and model comparison view

[Insert Figure / Screenshot Here]

Figure 5. Individual and batch student prediction workflow

4.4.4 Deployment Considerations

- The application can be run locally for demonstration/evaluation, or deployed to a cloud platform (e.g., Render, Railway, PythonAnywhere or a college server) for wider access, as noted in the Future Scope (Section 5.3).
- The saved model file is versioned (e.g., a model_version field) so that predictions can be traced back to the exact model that produced them.
- Access to the dashboard should be restricted to authorized staff accounts, consistent with the security requirement in Section 3.1 and the risk register in Section 3.7.

4.5 Testing

Table 11. Test plan and verification

Test Area	Test Type	Expected Outcome
Column validation (Module 1)	Unit test	Dataset missing a required column is flagged/rejected rather than silently processed
Missing value handling (Module 1)	Unit test	Rows/fields with missing values are handled per the defined strategy without crashing the pipeline
Train/test split (Module 2)	Unit test	Split preserves class proportions (stratified) and produces non-overlapping partitions
Random Forest training (Module 2)	Functional test	Model trains without error and produces predictions of the expected shape/type on test data
XGBoost training (Module 2)	Functional test	Model trains without error and produces predictions of the expected shape/type on test data
Metric computation (Module 2)	Unit test	Accuracy, Precision, Recall and F1-score are computed correctly against known test labels
Individual prediction (Module 3)	Functional test	Submitting a valid student record returns a prediction, risk level and contributing factors
Batch prediction (Module 3)	Functional test	Uploading a valid multi-row CSV returns a downloadable file with predictions for every row
Invalid input handling (Module 3)	Negative test	Malformed or incomplete input is rejected with a clear error message rather than causing a crash

Test Area	Test Type	Expected Outcome
End-to-end integration	Integration test	A record processed through Module 1 → Module 2 (trained model) → Module 3 produces a consistent, explainable result

4.6 Results and Discussion

Qualitatively, the ensemble models (Random Forest and XGBoost) are expected to outperform simple rule-based thresholds because they can capture interactions between attendance, marks, assignment completion and study hours rather than evaluating each factor in isolation. XGBoost is generally expected to match or exceed Random Forest on accuracy for structured tabular data of this kind, while Random Forest often remains competitive and slightly easier to interpret through its feature-importance ranking; the actual result should be confirmed against Table 10 once real experimental values are recorded. The dashboard's ability to surface contributing factors alongside each prediction directly supports the project's explainability requirement (Section 3.1) and mitigates the over-reliance risk identified in the risk register (Section 3.7).

4.7 Achievement of Objectives

Table 12. Achievement of objectives

Objective (from Section 1.5)	Status
Modular pipeline (collection → preprocessing → modelling → deployment)	Achieved – implemented as Modules 1, 2 and 3
Preprocessing workflow (validation, missing values, encoding, ID separation)	Achieved in Module 1
Train and compare Random Forest and XGBoost	Achieved in Module 2
Evaluate with Accuracy/Precision/Recall/F1 and select best model	Achieved in Module 2 (Section 4.3.4–4.3.5)
Web dashboard with exploration, comparison, analytics, individual & batch prediction	Achieved in Module 3
Testing across preprocessing, modelling and dashboard workflows	Achieved (Section 4.5); extended real-world validation left as future work

4.8 Project Management

Project Timeline

Table 13. Project timeline

Phase	Activities	Approx. Duration
Phase 1	Requirement analysis, literature review, dataset identification	[Insert duration]
Phase 2	Module 1 – Data collection & preprocessing	[Insert duration]
Phase 3	Module 2 – Model development, training and evaluation	[Insert duration]
Phase 4	Module 3 – Dashboard design and deployment	[Insert duration]
Phase 5	Testing, documentation and report preparation	[Insert duration]

Individual contributions and their approximate weighting are documented in the Individual Contribution Statement (front matter) and summarized again in Appendix F.

CHAPTER 5

CONCLUSION & REFLECTION

5.1 Conclusion

This project designed and implemented an AI-based system that predicts student academic performance from historical academic and behavioural data using machine learning. Random Forest and XGBoost classifiers were trained and compared using standard evaluation metrics, and the better-performing model was selected for deployment. The system identifies weak and at-risk students at an early stage, reports the key factors contributing to each prediction, and classifies students into risk categories to support timely intervention. A web-based dashboard makes dataset exploration, model comparison, analytics, and both individual and batch student prediction accessible to teachers and academic coordinators without requiring programming knowledge, turning a technically complex ML pipeline into a practical day-to-day decision-support tool.

5.2 Limitations

- Model performance depends heavily on the size, quality and representativeness of the available academic and behavioural dataset; results may not generalize to cohorts with different characteristics.
- Longitudinal outcome data – confirming whether flagged at-risk students actually improved after intervention – was not available at the time of writing, so the system's real-world impact has not yet been validated.
- Predictions are probabilistic estimates, not certainties, and should always be reviewed by a teacher before any academic decision or intervention.
- The current prototype uses static, uploaded datasets rather than a live feed from institutional systems.

5.3 Future Scope

- Real-Time Data: Integrate live attendance and academic-record feeds so predictions stay current without manual dataset uploads.

- Personalized Support: Generate tailored improvement suggestions for each student based on their specific contributing factors.
- Explainable AI: Extend the contributing-factors output with richer explanation techniques (e.g., SHAP values) to show precisely why a prediction was made.
- Performance Tracking: Track a student's predicted and actual performance across multiple semesters to monitor trends over time.
- Mobile Application: Provide a mobile-accessible version of the dashboard for on-the-go use by staff.
- Cloud Deployment: Deploy the system on a cloud platform for wider, more scalable institutional access.

5.4 Reflection

Working on this project strengthened my practical understanding of the full machine-learning lifecycle – from raw, imperfect data through preprocessing, comparative model training and evaluation, to serving a model through a usable web interface. Structuring the work across three modules (data preprocessing, model development, dashboard and deployment) gave each stage of the pipeline a clear, testable scope, while managing the interfaces between modules – particularly the handoff between the trained model in Module 2 and the prediction service in Module 3 – reinforced the value of modular, well-documented design. The exercise also reinforced the importance of treating a prediction as decision support rather than a verdict, since the system deals with real students and real academic outcomes.

REFERENCES

[1] Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32. Springer.

[2] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM.

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. JMLR.

[4] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference. SciPy.

[5] Abadi, M. et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX.

APPENDICES

Appendix A – Glossary of Terms

Term	Meaning
Machine Learning (ML)	Algorithms that learn patterns from data rather than following fixed, hand-written rules
Random Forest	An ensemble of decision trees whose individual predictions are combined (majority vote) to improve accuracy and reduce overfitting
XGBoost	A gradient-boosted decision-tree algorithm that builds trees sequentially to correct previous errors
Feature	An individual measurable input variable used by the model (e.g., attendance percentage)
Label	The known outcome/category a model is trained to predict (e.g., performance category)
Precision / Recall / F1	Standard classification metrics used to evaluate prediction quality (see Section 4.3.4)
Risk Level	A category (e.g., low, medium, high) summarizing a student's predicted likelihood of underperforming

Appendix B – Additional Functional Requirements / User Stories

- As a teacher, I want to upload a class dataset and immediately see which students are flagged as at-risk, so that I can plan interventions early.
- As an academic coordinator, I want to compare Random Forest and XGBoost results, so that I can trust the model behind the predictions.
- As a staff member, I want to predict a single student's outcome quickly, so that I can check a specific case during a parent meeting.
- As an administrator, I want batch predictions downloadable as a file, so that I can archive or share results with department heads.

Appendix C – Detailed Test Cases

Test ID	Description	Input	Expected Result
TC-01	Valid dataset upload	CSV with all required columns	Preprocessing completes; dataset ready for Module 2
TC-02	Missing column	CSV missing 'attendance' column	System flags missing column and rejects/requests correction
TC-03	Missing values in a row	Row with blank marks field	Missing value handled per defined strategy; no crash
TC-04	Model training	Prepared training set	Both RF and XGBoost models train and return metrics
TC-05	Individual prediction – valid input	Complete single-student form	Prediction, risk level and contributing factors returned
TC-06	Individual prediction – invalid input	Form with non-numeric marks field	Clear validation error shown; no server crash
TC-07	Batch prediction	CSV with 50 student records	Downloadable CSV returned with a prediction per row

Appendix D – Data Dictionary

Field	Example	Purpose / Notes
student_id	STU-00123	Unique identifier; excluded from ML features
name	Not used in modelling	Identifier only; excluded from ML features
attendance_percent	82.5	Behavioural feature – attendance rate
internal_marks	68	Academic feature – internal assessment score
assignment_score	75	Academic feature – assignment completion/score
study_hours_per_week	6	Behavioural feature – self-reported study hours
performance_label	At Risk / Average / Good	Target label used for training (historical outcome)
predicted_performance	At Risk	Model output for a given input record

Field	Example	Purpose / Notes
risk_level	High	Derived category (Low / Medium / High) from predicted_performance
model_version	v1.0	Identifies which saved model produced a given prediction

Appendix E – Ethics and Data-Handling Checklist

- Student data is used only for the stated purpose of academic performance prediction.
- Access to raw data and predictions is limited to authorized teaching/administrative staff.
- Predictions are presented as advisory, decision-support information, not as final judgements about a student.
- Contributing factors are shown alongside each prediction to keep the model's reasoning inspectable.
- Any future real-world deployment should be reviewed against the institution's data-protection policy before processing live student data.

Appendix F – Individual Contribution Evidence

Evidence of individual contribution includes: Module 1 preprocessing code and validation records; Module 2 model-training notebooks, evaluation output and comparison results; Module 3 dashboard source code, UI screens and deployment configuration; and this project report, prepared and reviewed in full by T. Niranjan Kumar. Evidence should accurately reflect completed work and should not include unsupported claims.

Appendix G – Outcome Mapping Sheet

The following mapping links each area of engineering evidence to its location in this report, for departmental/faculty assessment.

Outcome Evidence	Location in This Report
Problem formulation	Ch. 1
Literature review & engineering gap	Ch. 2

Outcome Evidence	Location in This Report
Engineering design under constraints	Ch. 3
Ethics, safety, sustainability, risk & security	Ch. 3 (3.7)
Detailed implementation – Module 2 (Model Development)	Ch. 4 (4.3)
Detailed implementation – Module 3 (Dashboard & Deployment)	Ch. 4 (4.4)
Testing and verification	Ch. 4 (4.5)
Results and discussion	Ch. 4 (4.6)
Project planning and management	Ch. 4 (4.8)
Individual contribution	Front matter + Appendix F
Conclusion, limitations and future scope	Ch. 5